

How Dragon Ball: The Breakers Matchmaking Works

A reverse-engineering explainer — what we've confirmed from decrypted packet captures, and what's still unknown. Compiled from the project docs vault ([Structure/Diarkis.md](#), [API/Calls/Matchmaking_Flow.md](#), [API/battle/*](#), [Structure/Version_Skew.md](#)) and the golden transcripts. State as of 2026-09-18 — **documentation roadmap complete (M0–M5, vault tag docs-m5-complete)**; 48 Diarkis sessions decrypted across three captures, zero residual decrypt failures.

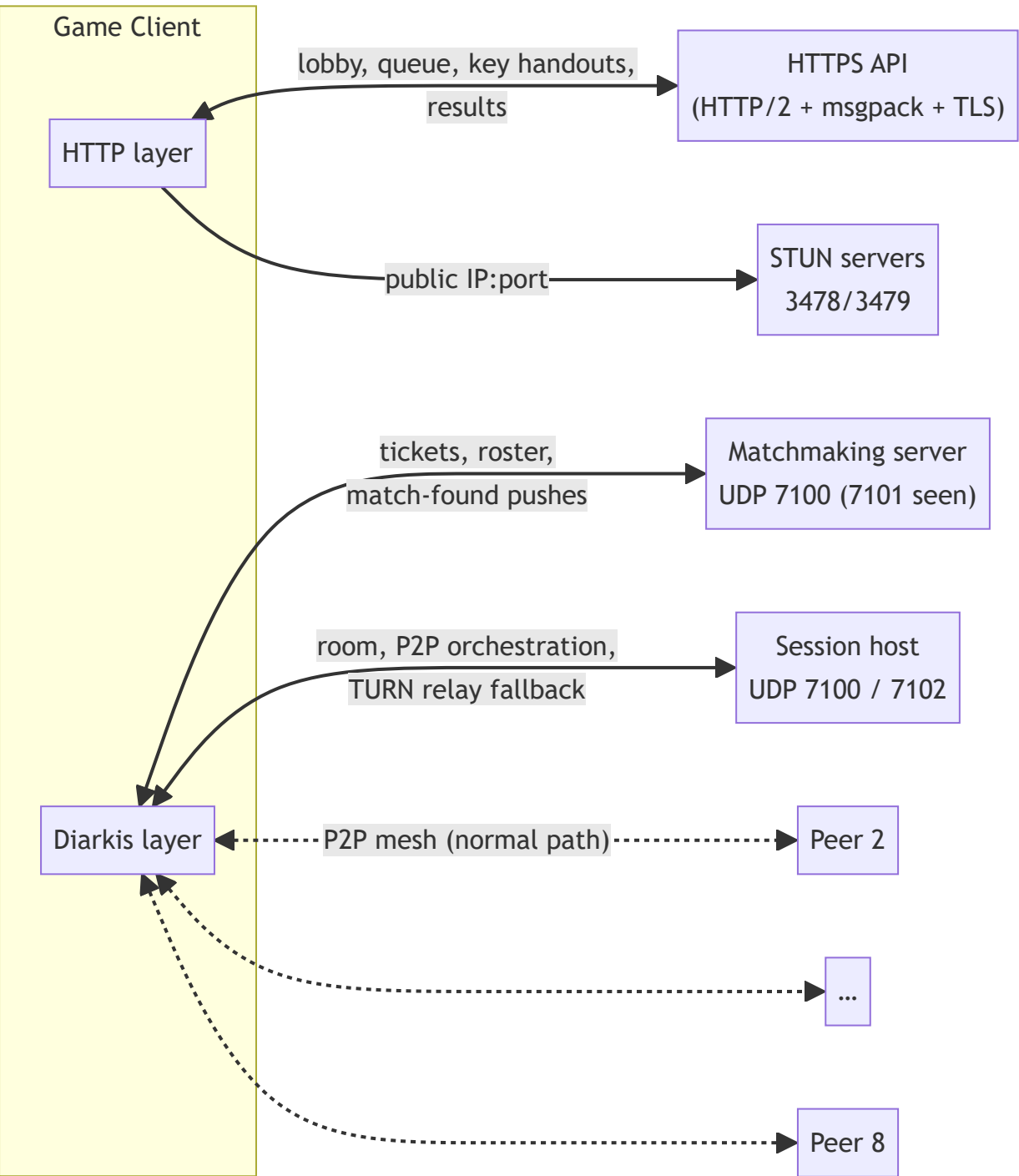
CONFIRMED verified byte-by-byte against decrypted captures **PARTIAL** structure decoded, details pending **UNKNOWN** not yet decoded

1. The big picture: two protocol layers

Matchmaking is split across **two completely different protocols**:

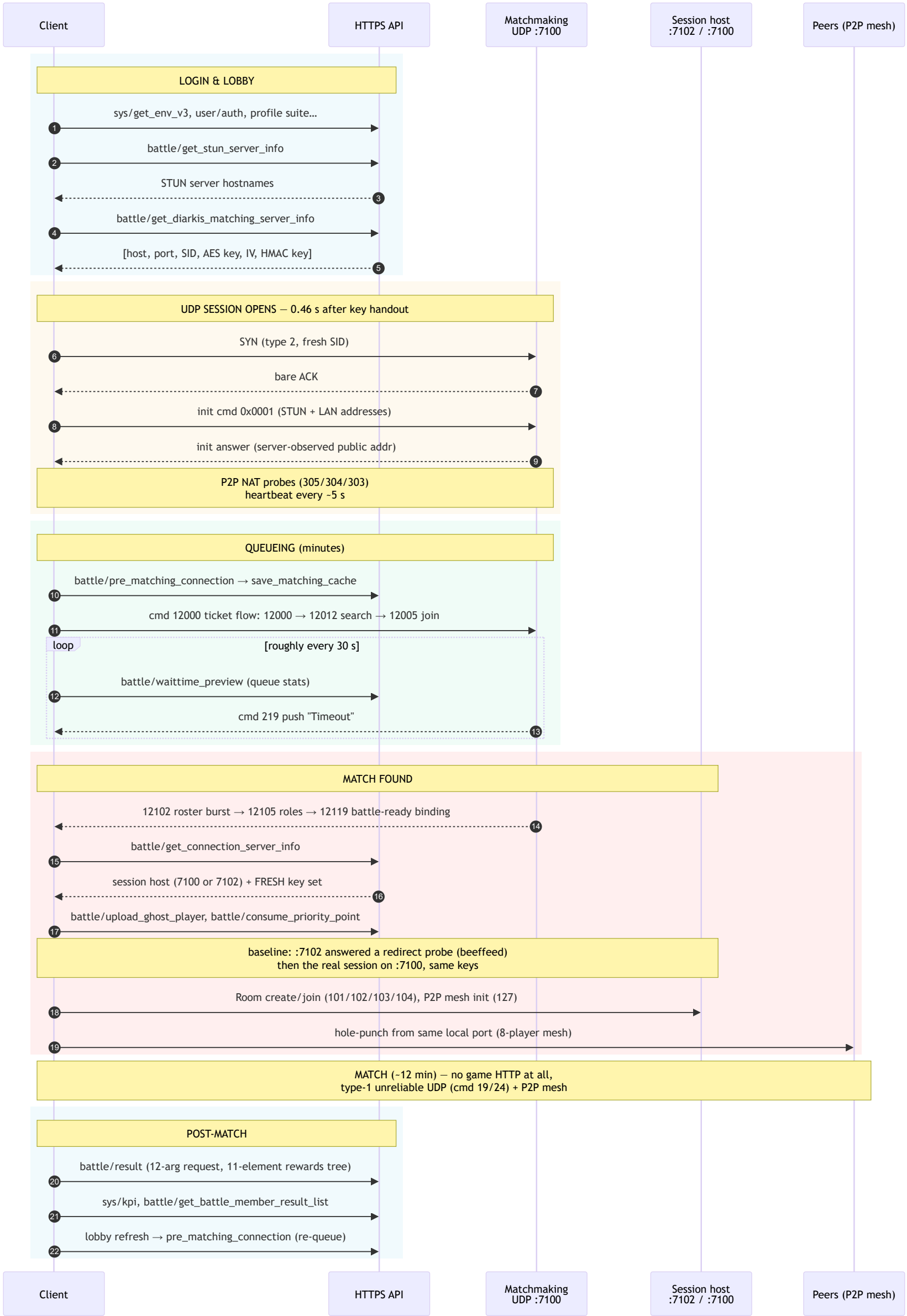
- **HTTPS API** (HTTP/2 + msgpack, TLS) — login, lobby state, queue management, and the crucial *key handouts*. This is the classic request/response layer.
- **Diarkis** — a commercial UDP middleware (proprietary Go server; Bandai is an official customer). Everything real-time — the matchmaking session, room membership, P2P orchestration, and in-match traffic — rides on Diarkis over UDP (matchmaking on 7100, sometimes 7101; session host on 7100 or 7102).

The HTTPS layer *orchestrates*; the UDP layer *executes*. The glue between them is a per-session key handout: the API tells the client where the Diarkis server is and gives it the encryption keys for that one session.



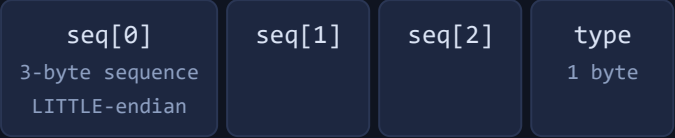
2. End-to-end: from login to match

Transcribed from three fully decrypted captures (`matchmaking data.pcapng` — 101 API calls — plus `matchmaking data 3.pcapng` and `more matches.pcapng` : 515 API calls total, three full matches). One subtle but important finding: **the UDP session opens at lobby entry, before the player even queues** — the SYN goes out 0.46 s after the key handout.



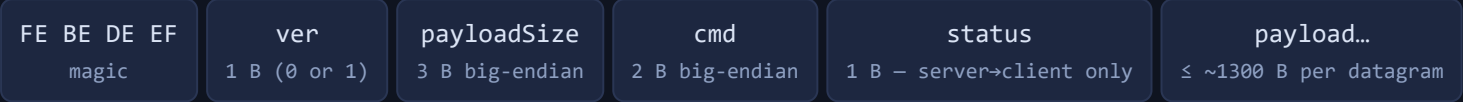
3. The Diarkis wire protocol

3.1 Datagram header — 4 bytes CONFIRMED



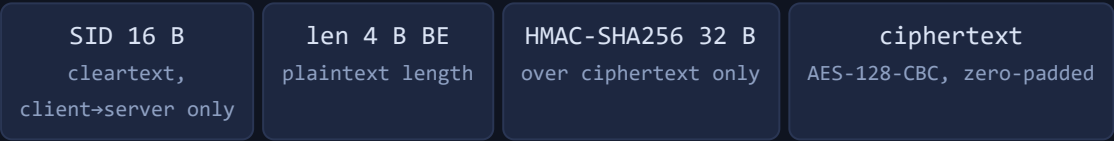
Type	Name	Role
1	UNRELIABLE	In-match data & P2P signaling, no ACK — dominates match traffic
2	SYN	Session open — 20 B: header + 16-byte session ID
3	DAT	Reliable data, carries a command envelope
4	ACK	Asymmetric: server = bare 4 B; client = N×20 B records with SID, batched (max 17)
5	RETRANSMIT	Resend of an unacked DAT every 0.5 s (NOT a reset — the old "RST" reading was wrong)
6	EACK	UNKNOWN never observed in 48 sessions
7	FIN	Teardown, 20 B; sequence field carries the sender's <i>next</i> DAT sequence

3.2 Command envelope CONFIRMED



Status byte: `01` OK · `04` BAD (3 envelopes across 2 streams, all "Missing room ID" on cmd 0x000b) · `ff` asynchronous push (very common) · `05` ERR never observed. Messages over ~1300 B are fragmented with a `FF FE FD FC` fragment header (largest reassembled group: 3727 B).

3.3 Crypto envelope CONFIRMED



- Fixed key and fixed IV per session — **no per-packet IV**. (The old `RUDPserver.py` `decryptData` treats bytes 32:48 as a per-packet IV — wrong, do not copy it.)
- Zero padding is always 1–16 bytes: a full extra block is added when already aligned. Senders can under-report the length field (cmd 0x0018 ext) — trust the HMAC, not the padding.
- 48 sessions decrypted across three captures with zero residual crypto failures**; the six early "decrypt-fails" all turned out to be decoder bugs, fixed in `diarkis_decode2.py`.

3.4 Where the keys come from CONFIRMED

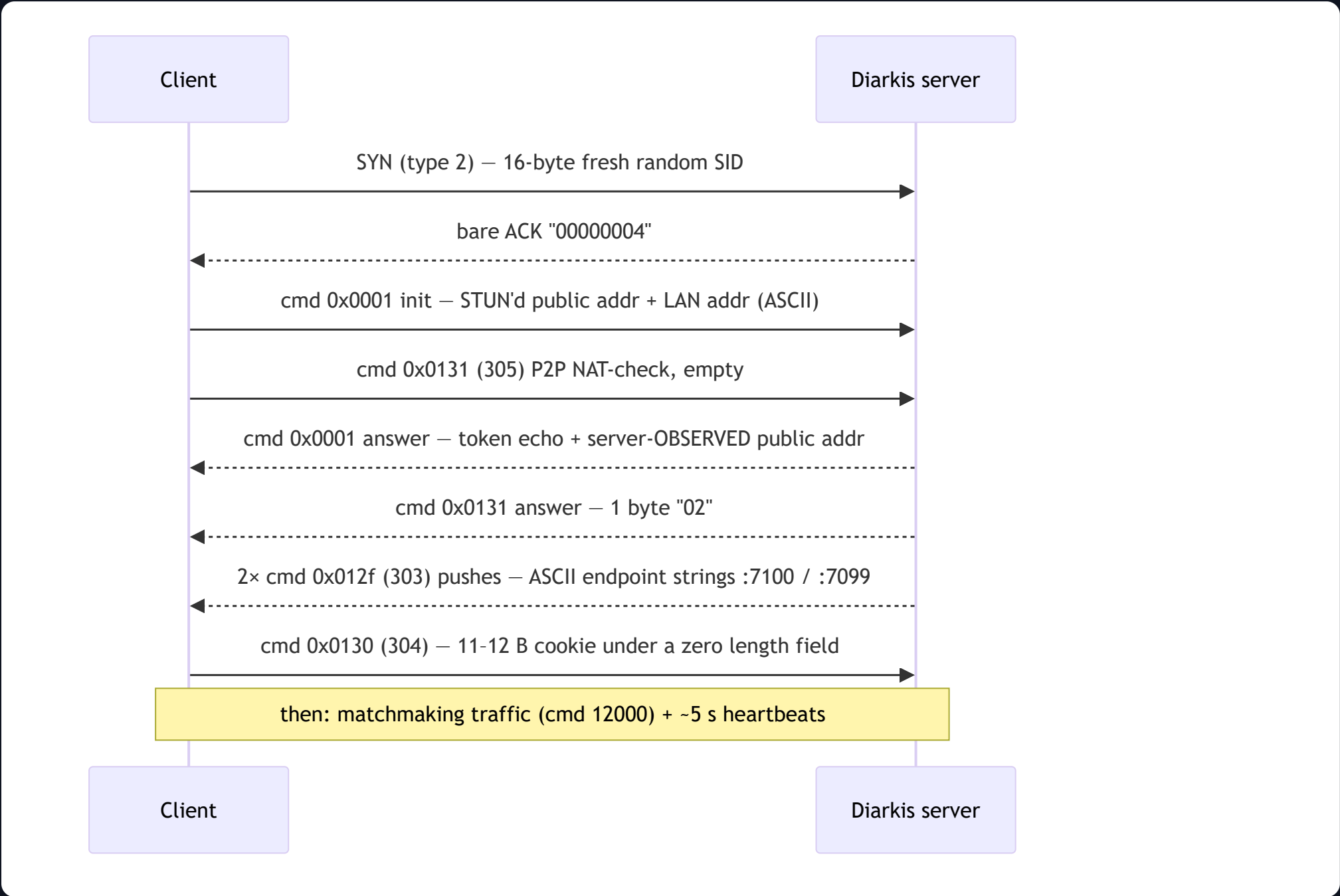
`get_diarkis_matching_server_info` (and `get_connection_server_info` for the session host) hands out everything positionally:

Response position	Wire role
<code>[1][0]</code>	Endpoint hostname (reversed-IP <code>.bc.googleusercontent.com</code>)
<code>[1][1]</code>	UDP port — matchmaking 7100 (7101 seen); session host 7100 or 7102 (7× vs 3× of 10 samples)
<code>[1][2]</code>	16-byte session ID (SID) — body of the SYN, cleartext prefix on every client DAT
<code>[1][3]</code>	AES-128-CBC key
<code>[1][4]</code>	AES-128-CBC IV (fixed for the session)
<code>[1][5]</code>	HMAC-SHA256 key (over ciphertext only)

Keys are per-session random — actually per-request random: 38 pairwise-distinct tuples observed, including three fresh sets issued within 0.85 s. Consequence for our server: the HTTP endpoint must generate each key set at runtime and share it with the UDP server process. There are no fixed keys to hardcode.

The leading `0` in these responses is the endpoint's result code (a convention shared by every `/025348/` response). The trailing `[1,1]` on the matchmaking handout is two u32 — constant across all 37 observed responses; semantics still unknown.

3.5 Session bootstrap CONFIRMED



Fun detail: the client's STUN-reported address and the address the server actually observes *differ* — the server truthfully reports what it sees, which is what later feeds P2P introduction. (Correction: cmd 304 is *not* empty — it carries a small cookie; the length field reads 0.)

3.6 The 0c0c directory service CONFIRMED

A separate lightweight **cleartext** protocol on the same port (no RUDP header, no encryption). The client sends a 26-byte query containing its SID; the server answers with a 51-byte packet carrying an ASCII endpoint. Both sides transmit in **bursts of 5 identical copies** on a ~1 s cadence — the 5× duplication seen in early capture analysis is specific to this service, not RUDP in general. The client probes every candidate server this way (20–30 packets per probe stream) while holding a full RUDP session with its chosen one.

```
query (C→S, 26 B): 0000 0c0c 00000001 0010 <16-byte SID>
answer (S→C, 51 B): 0f0f 0c0c 00000001 0029 "<reversed-ip>.bc.googleusercontent.com:7100"
```

UNKNOWN the flag-flip semantics (client `0000→0f0f`, server `0f0f→0000` after the first exchange).

4. The actual matchmaking conversation

CONFIRMED

Surprise finding (now fully verified): DBTB does *not* use Diarkis's public MatchMaker ticket commands (218–229) — byte-by-byte verification across 48 sessions shows they never appear in this build. Matchmaking rides a **custom command 12000** (`0x2ee0`) — a "CSMS" envelope carrying msgpack with an embedded JSON string. A second custom command **22000** (`0x55f0`) handles connection-room coordination. The only public ticket command ever observed is **219**, a push whose payload is literally the ASCII string `"Timeout"`, repeated while queued.

```
cmd 12000 payload framing (client→server):
  "CSMS09.01"          version string - ASCII "09"
  5 B zeros
  18-digit ASCII user ID      (server pads to 20 chars with a leading 0)
  16 B zeros
  u32be                  inner payload length
  u16be                  sub-ID
  msgpack map            {title_cd, platform, user_id, id, data} - data is a JSON string

server→client variant: binary u16be platform (not ASCII "09"),
pushes additionally prepend u32be = payload length - 4
```

4.1 The real ticket flow

CONFIRMED

Step	Sub-ID	Direction	Meaning
1	12000	C→S	Ticket session open
2	12012	C→S	Ticket create/search — <code>{"desiredRole":20,"desiredStageId":"unselected"}</code>
3	12005	C→S	Join room — <code>{"roomId":"...", "sdpData":{"player_level, p2p_uid, pf_uid (SteamID), patroller_name, able_rematch..."}}</code>
4	12017	C→S	Leave room
5	12102	S→C	Roster burst — battleRoomId + full roster (fragmented, up to ~3.7 KB)
6	12105	S→C	Role assignment (roleList is authoritative: 0 = Raider, 1 = Survivor)
7	12119	S→C	Battle-ready / connection-room binding (its advertised <code>serverIp:80</code> is never used)

Other observed sub-IDs: 12001/12002 c→s; 13xxx are the +1000 responses; pushes include 12101/12116/12118/12120. The connection-room command 22000 mirrors the pattern (22000/22001/22002/22004 c→s, 23xxx responses, pushes 22100/22101/22104/22105); **22002 carries per-peer RTT/TURN telemetry.**

4.2 Session host: rooms & P2P

CONFIRMED

Cmd	Role
101 (<code>0x0065</code>)	Room create/join — u32 + 52-char GUID; the leading u32 correlates with the HTTP session-token prefix (inferred)
102 (<code>0x0066</code>)	Attach/leave (GUID + user id), "OK" answers, member-left pushes
103 (<code>0x0067</code>)	16-byte member-ID register/distribute
104 (<code>0x0068</code>)	"RoomMessage" — first byte is a reliable flag. Two duties: property sync (8× 24-B slot bag) and the bulk in-match relay (~164k c→s / 1.21M s→c messages across three full matches; payloads 1–270 B c→s, 2–313 B s→c)

115 (0x0073)	Identity/rule exchange — GUID + userName + ~100 B JSON
127 (0x007f)	P2P mesh init — 784 B: 8× user id + base64 endpoint lists; client acks with 1 byte 00
301 (0x012d)	P2P-established report — empty payload (the 16 zero bytes are AES padding), one per peer
19 / 24	In-match relay channel on <i>type-1 unreliable</i> datagrams — exe names: RoomRelayProfile / RoomRelayToProfile

Gameplay is P2P: an 8-player mesh, hole-punched from the same local port the client uses for its session. P2P packet family 0b0e0e0d / 0d0e0a0f: type 0x01 sync (76–108 B, 92 B dominant), type 0x04 reliable (76–332 B), type 0x05 = the type-4 acknowledgment (one ~21 ms after every type-0x04 — common, not rare as first thought). **Relay-request 302 does not exist in this build** (unnamed in the cmd→name table, no senders/handlers); the relay fallback is game-level Corn TURN packets (types 29–32) through the session host — one captured session pushed ~69 MB through it. Our server mostly *orchestrates*; it rarely carries match traffic.

5. What we know ✓

- **CONFIRMED** Full HTTPS call order, login → queue → match-found → results → re-queue (frame-referenced, three captures)
 - **CONFIRMED** Rolling session token: each response's session becomes the next request's meta session
 - **CONFIRMED** All battle endpoint layouts — requests/responses for the 8 matchmaking-lifecycle endpoints + upload_ghost_player, with per-field confidence marks (M3)
 - **CONFIRMED** Complete Diarkis framing: datagram header, types, ACK asymmetry, retransmission, FIN semantics
 - **CONFIRMED** Crypto envelope end-to-end; 48 sessions decrypted, zero residual crypto failures
 - **CONFIRMED** Keys are per-request random; handout positions mapped to wire roles
- **CONFIRMED** Session bootstrap incl. server-observed-vs-STUN address difference and the cmd-304 cookie
 - **CONFIRMED** 0c0c directory wire format & the 5×-burst behavior being directory-only
 - **CONFIRMED** The real ticket flow (12000→12012→12005→12017 → 12102 → 12105 → 12119) and the full command census (M4)
 - **CONFIRMED** Port-7102 redirect probe (beeffeed) — one of two session-host port variants
 - **CONFIRMED** Working reference codec (diarkis_decode2.py) + 47 golden transcripts — the linchpin for the server reimplementaion
 - **CONFIRMED** TLS keylog coverage: all matchmaking HTTPS captures decrypt; http 2 data 3.pcapng identified as background noise (nothing recoverable)

6. What we don't know yet ?

Why it matters: so far we've only ever had to *read* the protocol — a server has to *write* it. Parsing something you don't fully understand is fine; generating it is where unknowns bite. The failure mode is asymmetric: everything we do know is byte-exact, so the server will work right up until a client does something we never captured. The remaining unknowns, grouped by how much trouble they cause:

Not really problems — stub-around-able

- **PARTIAL** `[1,1]` **handout tail** — meaning unknown, but constant across all 37 observed responses; our server can simply always send `[1,1]`. Unknown semantics, known behavior.
- **PARTIAL** **cmd-101 leading u32** — cosmetic correlation question (HTTP session-token prefix); a plausible generated value suffices.
- **UNKNOWN** **EACK (type 6), 0c0c flag-flip** — never-observed / edge behaviors; we mimic the observed pattern, nothing to get wrong.
- **PARTIAL** **P2P type 0x05 semantics** — established as the type-4 ACK, and it lives inside the client-to-client mesh. Our server orchestrates but never speaks the mesh protocol — it's the clients' problem, not ours.

Annoying but survivable — client tolerance likely

- **PARTIAL** **waittime_preview** 's **six queue-stat ints** — layout confirmed, meaning open (sentinel `0x7FFFFFFF`). The server generates these, so we send plausible values; worst case the queue ETA display looks wrong while matchmaking works.
- **UNKNOWN** **Member-result row int #2** — a finish/report status slot (mutates $-1 \rightarrow 8$ between polls), not placement. Post-match stats may display something odd.
- **PARTIAL** **battle/result** **token encryption layer** — tokens are client-computed; a preservation server can accept them without validating. Only matters for anti-cheat-style validation, which we don't do.

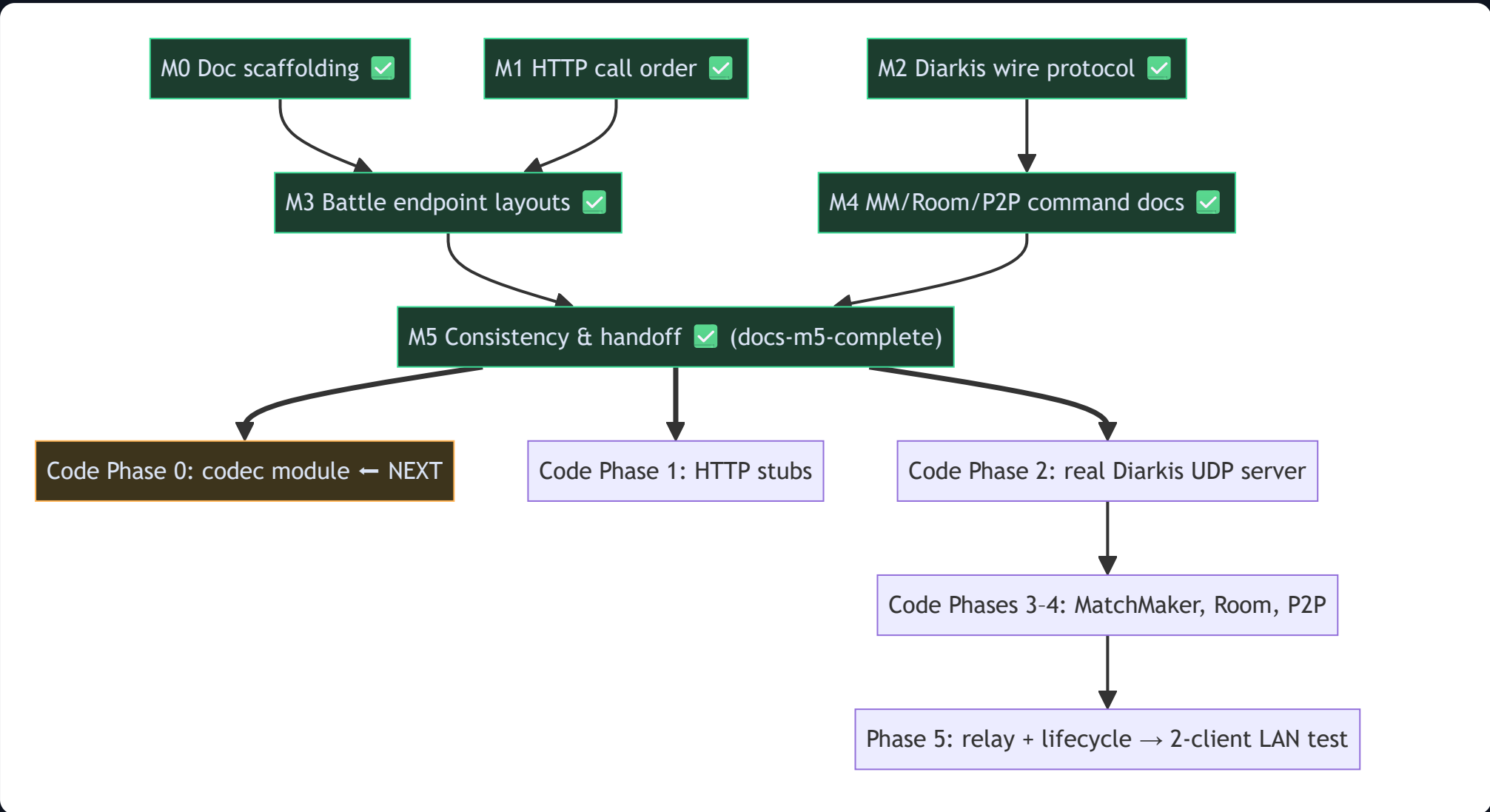
The real risks — unobserved flows

- **PARTIAL** **battle/start** **has exactly one wire sample** — layout is exe-confirmed and matches it, but we've never seen variance. If ranked/unranked or raider-initiated starts differ, we won't know until a client errors.
- **UNKNOWN** **Locked captures** (three `udp 7100 data*.pcapng` + one M4 stream — keys predate the keylog). They hold the flows we've never seen decrypted: **raider-side matchmaking, rematch, error paths**. Our docs describe the happy path for a survivor queueing normally.
- **UNKNOWN** **practice_*** / **orientation_*** **endpoints are stubs** — if the client calls these on first launch or tutorial entry, they need *something* faithful; currently unimplemented.
- **PARTIAL** **cmd 12116 + cmd-104 property-bag internals** — a push we don't understand, and the room-state sync (stage/character picks?) whose contents we can't parse. We can relay both opaquely — unless the client validates contents.

That's why the code plan ends with a **live two-client LAN test** (Phase 5): it's the moment the remaining unknowns stop being theoretical. The most valuable future captures, if the game is ever run with the keylog setup again: **raider-side queueing, a rematch flow, and a forced error** (e.g. cancel mid-queue) — those three would close most of the real-risk group.

RESOLVED for the record: relay-request 302 (absent from the build — Corn TURN instead) · CSMS push-header variant (binary u16be platform) · cmd-304 "empty" payloads (cookie) · stream 45 (cleartext DIR probes only) · `http 2 data 3.pcapng` (background noise) · leading `0` in handouts (endpoint result code).

7. Where the project stands



The documentation phase is complete. Every protocol fact the code roadmap needs is written down with capture-file + frame evidence, and the disagreements between sources (exe vs pcaps vs public Diarkis docs) are adjudicated in `Structure/Version_Skew.md`.

The strategic decision stands: the official Diarkis server is proprietary and version-matched to Bandai's deployment, so **we reimplement the Diarkis subset DBTB needs in Python** (Twisted), using the verified codec as reference and the captures as ground truth. The current `RUDPserver.py` only completes one client's bootstrap — it dispatches by datagram *length* instead of header, keeps state in globals, has a broken decrypt routine, and hardcodes keys that reality proved are per-request random. Its replacement gets built from the verified protocol facts above.

Sources: private docs vault (tag `docs-m5-complete`, corrections `033c16c`) · public docs [DB-Breakers-Legacy/DBTB-API-Documentation](#) · golden transcripts for collaborators: [dev-amto/private-logs](#) · `MATCH-MAKING-PLAN.md` — all facts traceable to capture file + frame in the docs vault.